

《云计算与大数据处理》

实验报告

实验名称	数据湖与质量契约（实验 02）
实验日期	2026 年 10 月 27 日
专业	计科
班级	3
学号	2410311322
姓名	陶志杰
指导教师	刘锦行

一、实验环境与身份

环境沿用实验 01 的 .venv，未额外安装。本章新用到 pyarrow 与 duckdb，分别用于读写 Parquet 和执行 SQL 对账。

项目	实测内容
操作系统 / Python	Windows 11 家庭版 / 3.12.9
解释器	D:\mumu\work\.venv\Scripts\python.exe
项目工作目录	D:\mumu\work（课程根目录）
代码仓库	https://github.com/add1223/online_data
pyarrow	25.0.1（写 Parquet）
duckdb	1.5.5（SQL 核对）
boto3 / moto	1.43.93 / 5.2.3（实验 01 用，本章不再起服务）
依赖检查	pip check 无冲突

本章的代码改动、运行证据、报告和截图都同步保存在 https://github.com/add1223/online_data 的 submissions/lab02 目录。



图 1　环境证据（学号、实验号 02、run_id 同屏）

身份登记还是先做。学号 2410311322，实验号 02。



图 2　学号登记运行证据

二、问题与数据

实验 01 只证明了文件被原样保存。本章要回答另一类问题：这份数据能否使用，清洗过程中拒收了什么、数量是否守恒，以及下游拿到的是否为同一份数据。

2.1 六行手算

先按契约逐行推演，不参考程序输出。六行数据为 a=10、b=20、a=10（完全重复）、c=-1、d=not-a-time、e=30。合法独特数为 a、b、e 三条，重复一条，

隔离两条，保留事件的 input_tokens 总量为 10+20+30=60。

到达顺序	event_id	input_tokens	判定
1	a	10	合法，保留
2	b	20	合法，保留
3	a	10	与第 1 行完全相同，计重复
4	c	-1	负数，隔离
5	d	40	时间 not-a-time，隔离
6	e	30	合法，保留

如果第 3 行改成 ID=a、Token=99，不能简单保留最后一条。它是同 ID 不同内容，属于冲突，按本章规则应停止发布并先定义更正政策。完整手算原稿在 submissions/lab02/hand_calculation.md。

2.2 输入数据与字段

输入仍为 data/raw/events.jsonl，共 20,102 行，由实验 01 生成，本章未重新生成。

字段	实测值	含义
valid_unique	20000	合法且唯一
duplicates	100	把前 100 条又复制了一遍
invalid	2	bad-negative 的 input_tokens 是 -1；bad-time 的时间是 not-a-time

字段	实测值	含义
raw_rows	20102	原始总行数
sha256	0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854	跟实验 01 的回执一致
seed	42	固定种子

三、三层结构与质量规则

数据分三层。Bronze 是原始字节，不动。Silver 清洗后写 Parquet。Gold 按服务和日期汇总。

层	产物	这一层做了什么
Bronze	data/raw/events.jsonl	原样保留，3,892,184 字节
Silver	data/silver/events.parquet	校验、隔离坏行、按 event_id 去重、转 Parquet，224,693 字节
Gold	data/gold/service_daily.parquet	按 service + 日期汇总请求数、token、耗时、错误数

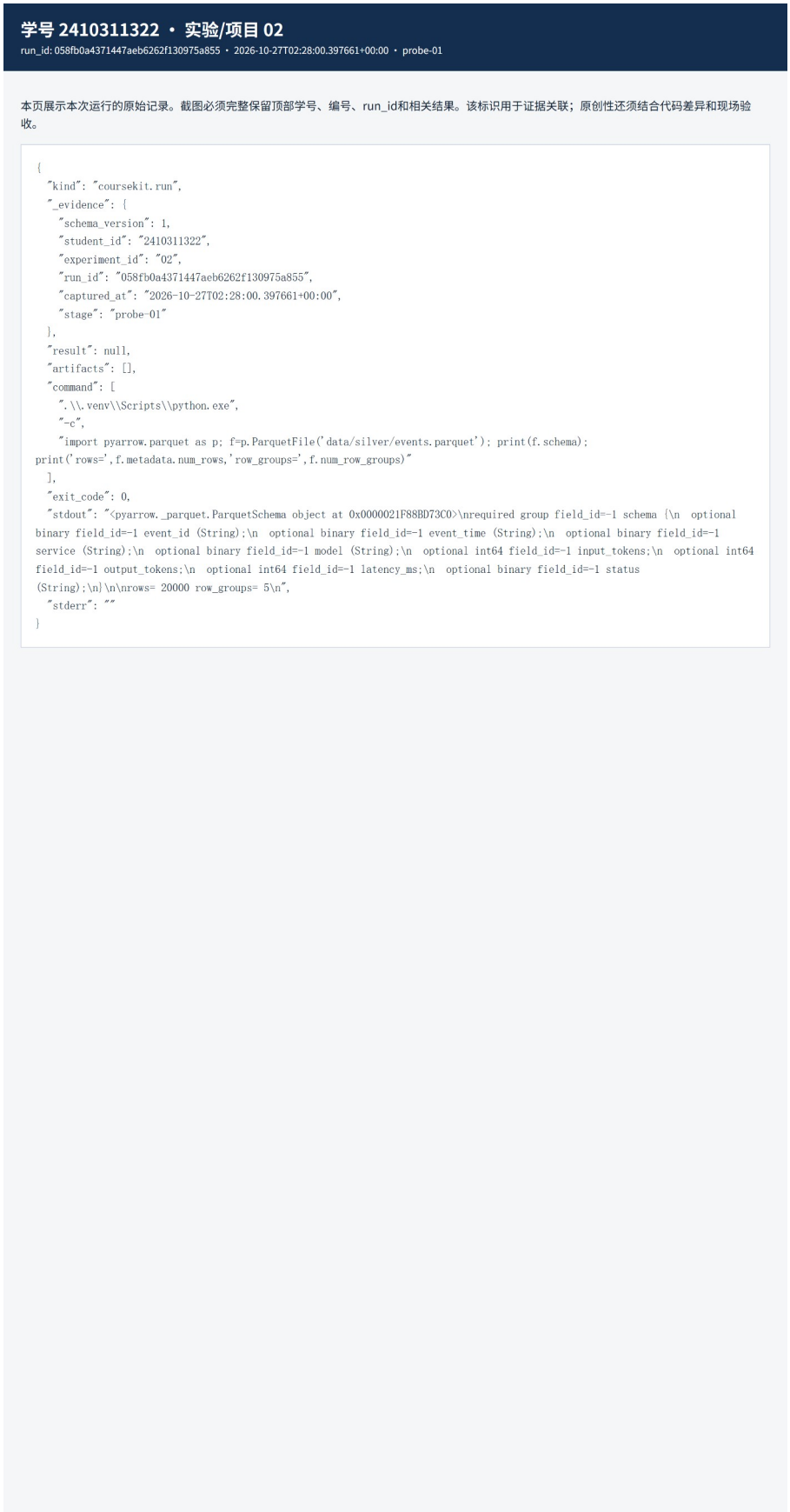
先把字段契约写清楚。下表是本章实际使用的 8 个字段、类型和约束：

字段	类型 / 约束	说明
event_id	字符串，唯一	事件的业务身份，重复按内容判断
event_time	ISO8601 字符串，UTC	必须带时区且偏移为 0，否则 non_utc_time

字段	类型 / 约束	说明
service	枚举 assistant/search/batch	只允许这三种服务
model	枚举 small/medium	模型档位
input_tokens	非负整数	type(x) is int，bool 和字符串都不接受
output_tokens	非负整数	同上
latency_ms	非负整数	单次请求耗时
status	枚举 ok/error	错误请求仍计入请求数和错误数

质量规则集中在 validate_event 里，一共四条：字段必须齐全；input_tokens 和 output_tokens 必须是非负整数；event_time 必须是带时区的 UTC 时间；同一 event_id 只能留一条。

这里有个细节值得单独说。判断整数用的是 type(x) is int，不是 isinstance。因为 Python 里 bool 是 int 的子类，用 isinstance 的话 True 会被当成 1 放进来。三个探针正好验证了这件事：negative 返回 invalid_input_tokens，boolean 也返回 invalid_input_tokens，naive_time 返回 non_utc_time。



学号 2410311322 · 实验/项目 02

run_id: 058fb0a4371447aeb6262f130975a855 · 2026-10-27T02:28:00.397661+00:00 · probe-01

本页展示本次运行的原始记录。截图必须完整保留顶部学号、编号、run_id和相关结果。该标识用于证据关联；原创性还须结合代码差异和现场验收。


```
{
  "kind": "coursekit.run",
  "_evidence": {
    "schema_version": 1,
    "student_id": "2410311322",
    "experiment_id": "02",
    "run_id": "058fb0a4371447aeb6262f130975a855",
    "captured_at": "2026-10-27T02:28:00.397661+00:00",
    "stage": "probe-01"
  },
  "result": null,
  "artifacts": [],
  "command": [
    ".\\.venv\\Scripts\\python.exe",
    "-c",
    "import pyarrow.parquet as p; f=p.ParquetFile('data/silver/events.parquet'); print(f.schema);
print('rows=',f.metadata.num_rows,'row_groups=',f.num_row_groups)"
  ],
  "exit_code": 0,
  "stdout": "<pyarrow._parquet.ParquetSchema object at 0x0000021F88BD73C0>\nrequired group field_id=-1 schema {\n  optional
binary field_id=-1 event_id (String);\n  optional binary field_id=-1 event_time (String);\n  optional binary field_id=-1
service (String);\n  optional binary field_id=-1 model (String);\n  optional int64 field_id=-1 input_tokens;\n  optional int64
field_id=-1 output_tokens;\n  optional int64 field_id=-1 latency_ms;\n  optional binary field_id=-1 status
(String);\n}\n\nrows= 20000 row_groups= 5\n",
  "stderr": ""
}
```


图 3　Parquet 分层结构探针（schema、行数、row group 数）

四、基线运行结果

coursekit lab02 跑完，回执里最重要的是守恒那一行。

指标	实测值	说明
raw_rows	20,102	原始总行数
valid_unique	20,000	清洗后保留
duplicate_rows	100	被去重
rejected_rows	2	被隔离
conservation	20,102	20000 + 100 + 2，等式成立
raw_bytes / parquet_bytes	3,892,184 / 224,693	Parquet 约为原始文件的 5.8%

按服务拆开的汇总：

service	requests	errors	input_tokens
assistant	14,008	543	12,631,850
search	3,969	179	3,567,223
batch	2,023	78	1,816,567
合计	20,000	800	18,015,640

三个服务合计 20,000 条，与 Silver 行数一致。错误数合计 800，相当于每 25 条出现一条错误，与生成规则相符。

学号 2410311322 · 实验/项目 02

run_id: 9e0807d7ce9d41af8e27514ffa909aff · 2026-10-27T02:27:39.487675+00:00 · verify

本页展示本次运行的原始记录。截图必须完整保留顶部学号、编号、run_id和相关结果。该标识用于证据关联；原创性还须结合代码差异和现场验收。


```
{
  "kind": "coursekit.run",
  "_evidence": {
    "schema_version": 1,
    "student_id": "2410311322",
    "experiment_id": "02",
    "run_id": "9e0807d7ce9d41af8e27514ffa909aff",
    "captured_at": "2026-10-27T02:27:39.487675+00:00",
    "stage": "verify"
  },
  "result": {
    "raw_rows": 20102,
    "valid_unique": 20000,
    "duplicate_rows": 100,
    "rejected_rows": 2,
    "conservation": 20102,
    "raw_bytes": 3892184,
    "parquet_bytes": 224693,
    "aggregate": [
      {
        "service": "assistant",
        "requests": 14008,
        "input_tokens": 12631850,
        "output_tokens": 4310029,
        "latency_ms_sum": 28448309,
        "errors": 543
      },
      {
        "service": "batch",
        "requests": 2023,
        "input_tokens": 1816567,
        "output_tokens": 616515,
        "latency_ms_sum": 4079006,
        "errors": 78
      },
      {
        "service": "search",
        "requests": 3969,
        "input_tokens": 3567223,
        "output_tokens": 1219318,
        "latency_ms_sum": 8137035,
        "errors": 179
      }
    ]
  },
  "artifacts": [
    {
      "file": "outputs/lab02/quality.json",
      "sha256": "716c419d1b509615a931b31a8875bd84e127f63f20a4a83dfb2208d6af9199e0",
      "_evidence": {
        "schema_version": 1,
        "student_id": "2410311322",
        "experiment_id": "02",
        "run_id": "9e0807d7ce9d41af8e27514ffa909aff",
        "captured_at": "2026-10-27T02:27:39.487675+00:00",
        "stage": "quality"
      }
    },
    {
      "file": "outputs/lab02/quarantine.json",
      "sha256": "edd0e32908a941403e2211c2ddecd5e02839eeb989560614956fb566877464b8",
      "_evidence": {
        "schema_version": 1,
        "student_id": "2410311322",
        "experiment_id": "02",
        "run_id": "9e0807d7ce9d41af8e27514ffa909aff",
        "captured_at": "2026-10-27T02:27:39.487675+00:00",
        "stage": "quarantine"
      }
    }
  ]
}
```


图 4　基线回执（守恒、字节数、三服务汇总）

4.1 存储与查询计划观察

Parquet 文件有 20,000 行、8 列，按 row_group_size=4096 写入，实际形成 5 个行组。查询计划是对 service='search' 求 input_tokens 总和：只扫描 1 个文件，投影列是 input_tokens，过滤条件是 service='search'，扫描 3,969 行，总时间 0.0279 秒。

观察项	实测	能说明什么
schema	8 列，event_id/event_time 为 string，token/latency 为 int64	字段类型与契约一致
row group	5 个，每组上限 4,096 行	行组是文件内部组织，不是目录分区
列裁剪	计划只投影 input_tokens	查询没有读取无关列
谓词下推	过滤条件 service='search' 出现在 TABLE_SCAN	过滤被交给扫描层，但未观察到行组跳过
目录分区	只有单个 events.parquet，未按日期或服务分目录	不能宣称实现了 Hive 目录分区

还要说明大小比较的局限。原始 JSONL 含 100 条重复和 2 条坏行，未压缩；Silver 已清洗并用 Zstd 压缩。3,892,184 字节对 224,693 字节，只能说明这份清洗产物的存储占用，不是“只换文件格式”的严格性能对照。

完整查询计划保存在 submissions/lab02/artifacts/query-plan.txt。

五、个人改动：阈值 2000 → 1200

5.1 改了什么

复制附件到 submissions/lab02，只改一个常量。

```
-MAX_INPUT_TOKENS = 2000 +MAX_INPUT_TOKENS = 1200
```

完整 diff 在 submissions/lab02/quality_policy.diff。这个阈值是附加政策，不是基础校验。基线定 2000 时，所有合法事件的输入量都不超过 1800，所以它一条都没拒。改成 1200 之后才真正起作用。

规则	原始	合法独特	重复	拒绝	input_tokens 总量
基础课程契约	20,102	20,000	100	2	18,015,640
附加上限 2000	20,102	20,000	100	2	18,015,640
附加上限 1200	20,102	13,405	67	2 + 6,628	8,110,488

5.2 预测与实测

预测是在改成 1200 之前写下来、存进 submissions/lab02/预测原稿.md 的，带时间戳。

项目	预测	实测	结果
合法独特数	变少	13,405（原 20,000）	命中
input_tokens_sum	变少	8,110,488（降 55%）	命中
重复数	不变	67（原 100）	未命中
拒绝原因	2 类	3 类，新增 policy_input_limit 6,628 条	未命中

守恒还是成立的：13,405 + 67 + 2 + 6,628 = 20,102。

5.3 为什么重复数也变了

我原以为重复数只与 event_id 有关、与 token 阈值无关，因此预测它不变实际减少了 33 条。

原因是程序先校验、再去重。那 100 条重复行是前 100 条事件的副本，其中一部分 input_tokens 超过 1200，在 1200 阈值下先被 policy_input_limit 拒绝，不会进入去重环节。因此重复数从 100 降到 67，拒绝原因也多出一类。

这一点需要在报告中说明：拒绝数不能简单理解为「坏数据 2 条」。阈值越严格，被政策挡下的合法事件也会计入拒绝统计。

学号 2410311322 · 实验/项目 02

run_id: 1fad4d8d638f4cabbc009defbfae84dc · 2026-10-27T02:28:18.945520+00:00 · student_quality_policy

本页展示本次运行的原始记录。截图必须完整保留顶部学号、编号、run_id和相关结果。该标识用于证据关联；原创性还须结合代码差异和现场验收。


```
{
  "kind": "coursekit.run",
  "_evidence": {
    "schema_version": 1,
    "student_id": "2410311322",
    "experiment_id": "02",
    "run_id": "1fad4d8d638f4cabbc009defbfae84dc",
    "captured_at": "2026-10-27T02:28:18.945520+00:00",
    "stage": "student_quality_policy"
  },
  "result": null,
  "artifacts": [
    {
      "file": "outputs/lab02/student-policy-2000.json",
      "sha256": "df115d022931b92000d99707784693d4745c9e28b440cc145a51adfec613a44a",
      "_evidence": {
        "schema_version": 1,
        "student_id": "2410311322",
        "experiment_id": "02",
        "run_id": "1fad4d8d638f4cabbc009defbfae84dc",
        "captured_at": "2026-10-27T02:28:18.945520+00:00",
        "stage": "student_quality_policy"
      }
    }
  ],
  "exit_code": 0,
  "stdout": "{\n  \"max_input_tokens\": 2000,\n  \"raw_rows\": 20102,\n  \"valid_unique\": 20000,\n  \"duplicates\": 100,\n  \"rejected_by_reason\": {\n    \"invalid_input_tokens\": 1,\n    \"invalid_event_time\": 1\n  },\n  \"input_tokens_sum\": 18015640,\n  \"probe_errors\": {\n    \"negative\": \"invalid_input_tokens\",\n    \"boolean\": \"invalid_input_tokens\",\n    \"naive_time\": \"non_utc_time\"\n  },\n  \"_evidence\": {\n    \"schema_version\": 1,\n    \"student_id\": \"2410311322\",\n    \"experiment_id\": \"02\",\n    \"run_id\": \"1fad4d8d638f4cabbc009defbfae84dc\",\n    \"captured_at\": \"2026-10-27T02:28:18.945520+00:00\",\n    \"stage\": \"student-policy-2000\"\n  }\n}\n",
  "stderr": ""
}
```


图 5　阈值 2000 的个人脚本结果

学号 2410311322 · 实验/项目 02

run_id: 5255490bd1524ed984a9aaedacd7d463 · 2026-10-27T02:36:03.047708+00:00 · student_quality_policy

本页展示本次运行的原始记录。截图必须完整保留顶部学号、编号、run_id和相关结果。该标识用于证据关联；原创性还须结合代码差异和现场验收。


```
{
  "kind": "coursekit.run",
  "_evidence": {
    "schema_version": 1,
    "student_id": "2410311322",
    "experiment_id": "02",
    "run_id": "5255490bd1524ed984a9aaedacd7d463",
    "captured_at": "2026-10-27T02:36:03.047708+00:00",
    "stage": "student_quality_policy"
  },
  "result": null,
  "artifacts": [
    {
      "file": "outputs/lab02/student-policy-1200.json",
      "sha256": "2a85cf1bf0d0cc20e50ddeea56950a57ed27e2906670115b09fcb5e91276731c",
      "_evidence": {
        "schema_version": 1,
        "student_id": "2410311322",
        "experiment_id": "02",
        "run_id": "5255490bd1524ed984a9aaedacd7d463",
        "captured_at": "2026-10-27T02:36:03.047708+00:00",
        "stage": "student_quality_policy"
      }
    }
  ],
  "exit_code": 0,
  "stdout": "{\n  \"max_input_tokens\": 1200,\n  \"raw_rows\": 20102,\n  \"valid_unique\": 13405,\n  \"duplicates\": 67,\n  \"rejected_by_reason\": {\n    \"policy_input_limit\": 6628,\n    \"invalid_input_tokens\": 1,\n    \"invalid_event_time\": 1\n  },\n  \"input_tokens_sum\": 8110488,\n  \"probe_errors\": {\n    \"negative\": \"invalid_input_tokens\",\n    \"boolean\": \"invalid_input_tokens\",\n    \"naive_time\": \"non_utc_time\"\n  },\n  \"_evidence\": {\n    \"schema_version\": 1,\n    \"student_id\": \"2410311322\",\n    \"experiment_id\": \"02\",\n    \"run_id\": \"5255490bd1524ed984a9aaedacd7d463\",\n    \"captured_at\": \"2026-10-27T02:36:03.047708+00:00\",\n    \"stage\": \"student-policy-1200\"\n  }\n}\n",
  "stderr": ""
}
```


图 6　阈值 1200 的个人脚本结果

六、血缘与清单核验

data/manifests/silver.json 记了三处 sha256：原始文件、Silver Parquet、Gold Parquet。校验脚本逐个比对，结果输出 manifest hashes match。

这一步的意义在于：下游可以随时确认手中的 Silver 是否由这份原始数据计算得出；任何一层被替换，哈希都会立即对不上。

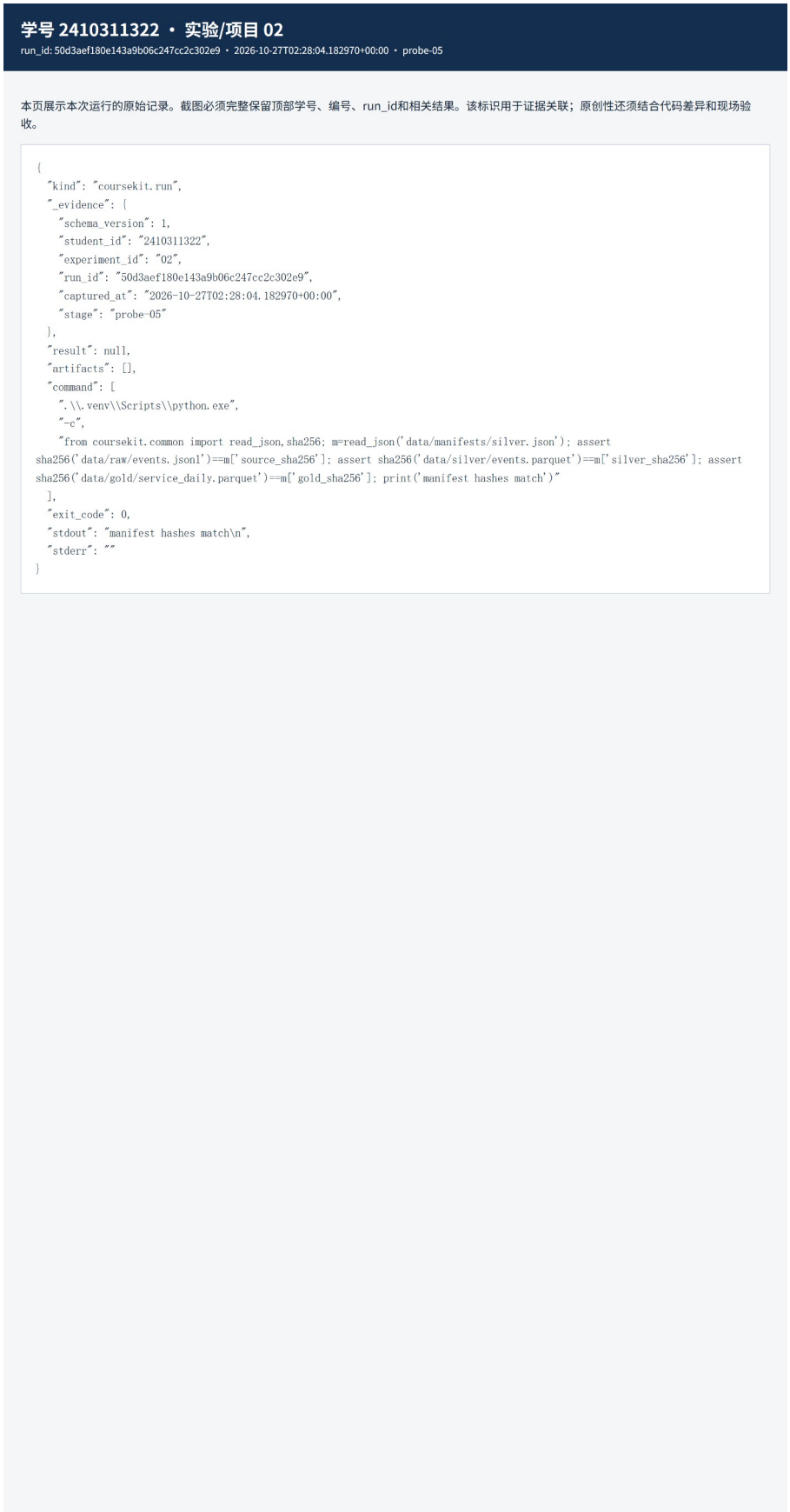


图 7　清单三处 sha256 校验通过

6.1 契约拒绝与修复

构造一条 input_tokens 为字符串 "100" 的记录，交给真实的 validate_event。它返回 invalid_input_tokens，因为契约只接受整数，不做静默转换。这条记录被拒绝，不进入 Silver。

```
mode= bad input_tokens= '100' validate_event= invalid_input_tokens CONTRACT_REJECTED invalid_input_tokens
```

修复不是放宽契约去接受字符串，而是修正样本本身，把 "100" 写成整数 100。同一探针再跑一次，validate_event 返回 None，记录被接受。这样既保留了严格契约，也验证了拒绝和恢复路径。

```
mode= fixed input_tokens= 100 validate_event= None CONTRACT_RECOVERED accepted
```

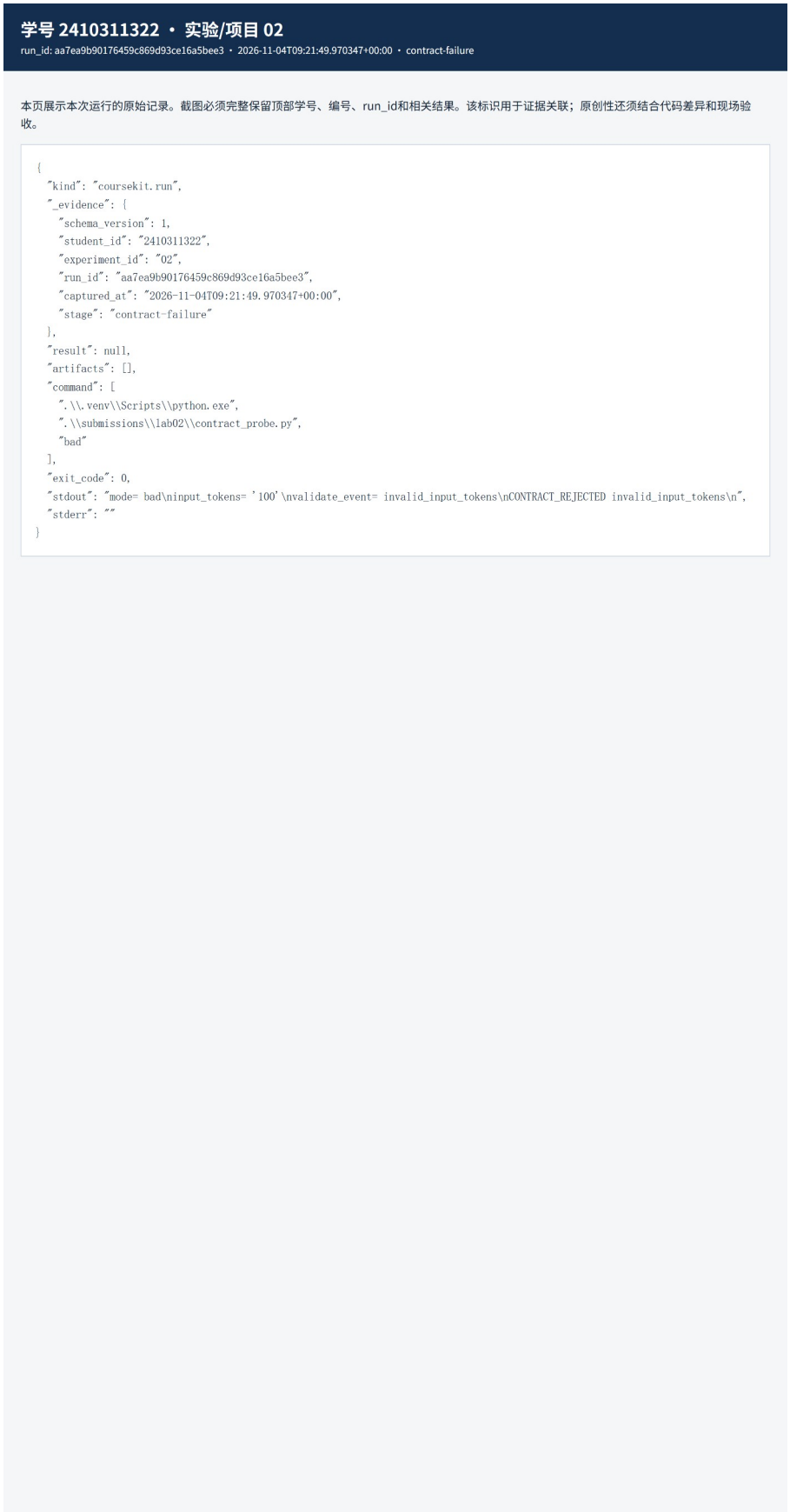


图 8　字符串 token 被契约拒绝（invalid_input_tokens）

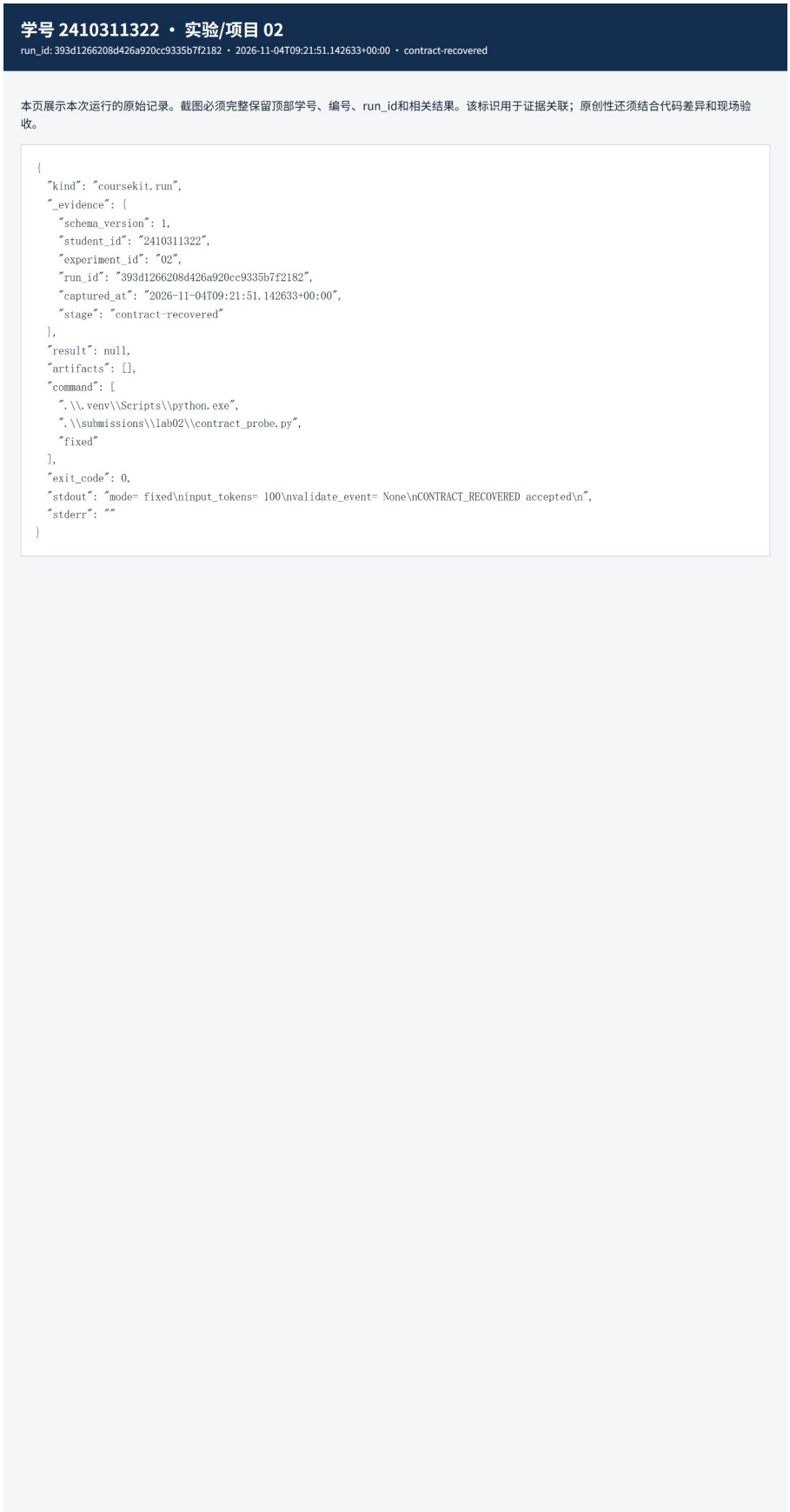


图 9　改为整数后通过校验（CONTRACT_RECOVERED）

如果原始文件本身被替换或损坏，不能修改清单摘要使其通过。正确做法是先判断是上游原始文件变化还是 Silver/Gold 损坏：原始文件变化就回到实验 01 重新生成并核验，再重跑实验 02；只有下游产物损坏则直接重跑实验 02。

七、能力边界

本章验证与未验证的范围分开说明。

项目	状态	依据
分层产物真实生成	验证	Silver 219 KB、Gold 1.8 KB 写在磁盘上，schema 与行数都读过
数量守恒	验证	20102 = 20000 + 100 + 2，个人改动后同样成立
质量规则生效	验证	三个探针分别被拒，且原因分类正确
清单哈希一致	验证	三处 sha256 全部匹配
去重的业务正确性	部分验证	只验证了同一 event_id 内容相同的情况；ID 相同但内容冲突时如何处理，本章未定义
跨机容灾	未验证	全部在本机单进程完成，没有多节点或故障注入
真实数据规模	未验证	数据是 2 万条合成记录，不代表生产量级下的性能

7.1 必答分析题

（1）True、"100" 与整数 100 的处理不同。契约用 type(x) is int，True 虽

然是 int 的子类也会被拒绝；"100" 是字符串，不做静默转换；只有整数 100 通过。

（2）同 ID 内容冲突不能当普通重复。重复是内容完全相同；内容不同说明同一业务身份出现两个版本，直接计数会掩盖冲突，本程序选择报错并停止发布。

（3）JSONL 与清洗 Parquet 的大小比较不是纯格式性能对照。输入含重复和坏行、未压缩，输出已清洗并用 Zstd 压缩，两者内容与编码都不同。

（4）列裁剪是只读需要的列；谓词下推是把过滤交给扫描层；行组是文件内部的块；目录分区是按目录拆文件。本章实现了列裁剪与过滤下推，设置了 4,096 行组，未做目录分区。

（5）阈值 1200 会丢掉 input_tokens 在 1201—1800 之间的合法业务请求技术校验判断记录是否符合类型和范围，业务政策决定哪些合法请求可以接收，两者要分开写。

（6）文件清单不等于 ACID 事务或完整湖仓表格式。清单只记录文件哈希和契约版本，没有事务隔离、回滚、并发控制和快照版本；它能支持血缘核

对，但不能替代数据库或表格式。

八、总结与交接

本章完成的工作：将原始数据划分为 Bronze/Silver/Gold 三层；用四条规则清洗并隔离坏行；按 event_id 去重；把结果写成 Parquet；核对数量守恒与服务汇总；修改输入阈值并做对照；核验三处 sha256。

最主要的收获并非 Parquet 比 JSONL 小多少，而是明确了校验与去重的顺序会影响统计口径：顺序一变，同一份数据的「重复数」就不再是同一个数。

8.1 交给后续章节的东西

交接内容	路径	谁会用
Silver 明细	data/silver/events.parquet	实验 03 的聚合、实验 04 的基础数据
Gold 日汇总	data/gold/service_daily.parquet	实验 07 的 /stats
三层清单	data/manifests/silver.json	实验 08 的 artifact_integrity 与血缘检查
个人对照结果	outputs/lab02/student-policy-1200.json	本章报告的证据

还没处理的问题：同一 event_id 但字段内容不一致时该怎么裁定，本章没

有定义；Gold 只按服务和时间汇总，没有区分模型维度；阈值 1200 是人为设定，没有业务依据。

8.2 运行记录索引

报告里每张图对应的运行记录如下。按 run_id 可以在 evidence/ 里找到原始 JSON 和同名 HTML。

图	run_id
图 1	445511dc35ff4228bb910ed11d716af1
图 2	ef0914c0c9c24d8d858729fcebfcf967
图 3	058fb0a4371447aeb6262f130975a855
图 4	9e0807d7ce9d41af8e27514ffa909aff
图 5	1fad4d8d638f4cabbc009defbfae84dc
图 6	5255490bd1524ed984a9aaedacd7d463
图 7	50d3aef180e143a9b06c247cc2c302e9
图 8	aa7ea9b90176459c869d93ce16a5bee3
图 9	393d1266208d426a920cc9335b7f2182

九、AI 使用披露

这次用了 AI 助手，主要用来解释 pyarrow 的 schema 输出、核对守恒算式、整理报告排版。

四项预测是我在改代码之前写的，两条猜错的地方也是我自己核对的。AI 提示我注意「先校验后去重」的顺序，我看了实际数字 67 和 100 的差异才确认这个解释成立。

所有数字都来自 outputs/lab02 下的真实 JSON，没有一处是生成的。